

TechStar

Electronics Repair Workshop
Management System

Contents

1	System Analysis	3
1.1	Introduction	3
1.2	Problem Statement	3
1.3	Objectives	4
1.4	System Users	4
1.5	System Assumptions	5
1.6	System Scope	5
1.7	Functional Requirements	6
1.7.1	Service Request Management	6
1.7.2	Search and Filtering	6
1.7.3	Staff Management	7
1.7.4	Real-Time Service Request Updates	7
1.7.5	Staff Group Chat and Notifications	7
1.8	Non-Functional Requirements	7
1.8.1	Performance	8
1.8.2	Scalability	8
1.8.3	Reliability	8
1.8.4	Security	8
1.8.5	Maintainability	8
1.8.6	Availability	9
1.8.7	Usability	9
1.8.8	Responsiveness	9
2	System Design	10
2.1	System Architecture	10
2.2	High-Level Architecture Diagram	10
2.3	Database Design	11

2.3.1	Database Schema	12
2.4	Entity-Relationship Diagram	12
2.5	API Design	14
2.6	WebSocket Communication	16
2.6.1	Client-to-Server Events	16
2.6.2	Server-to-Client Events	17
2.7	Background Processing	17
2.8	Technology Stack	17
2.9	Technology Selection Rationale	18
3	Implementation	19
3.1	Implementation Overview	19
3.2	Project Directory Structure	19
3.3	Backend Structure	20
3.4	Frontend Structure	21
3.5	Database Implementation	21
3.6	Authentication and Authorization	21
3.7	Customer Access Control	22
3.8	Real-Time Communication	22
3.9	Background Processing	22
3.10	API and Service Communication	23
3.11	Running the Application	23
3.11.1	Backend	23
3.11.2	Frontend	23
3.12	Implementation Summary	23

Chapter 1

System Analysis

1.1 Introduction

TechStar is a real-time electronics repair workshop management system designed to organize and streamline the service request lifecycle within an electronics repair workshop. The system provides dedicated functionality for supervisors, operators, technicians, and customers.

The platform combines service request management, customer self-service tracking, technician workload management, real-time communication, notifications, analytics, and background task processing into a centralized system.

The system is designed around a structured service request lifecycle and uses REST APIs, Socket.IO real-time communication, PostgreSQL database services, and background workers to coordinate operational activities.

1.2 Problem Statement

Traditional electronics repair workshops may rely on fragmented ticket tracking, manual status updates, paper-based records, and phone-based communication. These practices can make it difficult to maintain accurate service information and provide timely visibility into ongoing repairs.

The major problems addressed by TechStar are:

- **Customer Status Visibility:** Customers do not have a self-service mechanism for checking the current status of their repair requests, which can result in repeated phone inquiries.
- **Service Request Management:** Operators require a centralized mechanism for creating, searching, and monitoring repair requests.
- **Technician Workload Management:** Workshop staff require visibility into technician availability, workload, specialization, and capacity before assigning repair requests.

-
- **Supervisory Monitoring:** Supervisors require centralized dashboards for monitoring service requests, staff workload, revenue-related information, and operational performance.
 - **Staff Communication:** Staff communication can become fragmented when it is performed through external communication channels without direct association with the workshop management system.
 - **Background Processing:** Tasks such as overdue detection and email notification require reliable background processing without blocking normal API operations.

1.3 Objectives

The primary objectives of TechStar are:

1. **Automate the Service Request Lifecycle:** Provide a structured workflow for managing service requests through the following states:
RECEIVED → ASSIGNED → REPAIRING → WAITING_FOR_PARTS
→ READY_FOR_DELIVERY → COMPLETED
with additional terminal states including FAILED and CANCELLED.
2. **Enable Customer Self-Service:** Allow customers to check repair information using the Service Request ID and a unique 6-digit Customer Access Code without requiring application login.
3. **Improve Staff Workload Management:** Allow supervisors and operators to monitor technician capacity, availability, specialization, and active workloads when assigning service requests.
4. **Provide Real-Time Synchronization:** Use Socket.IO to distribute service request changes, technician assignment changes, chat messages, and notifications to connected clients in real time.
5. **Provide Management Analytics:** Provide time-based reporting and dashboard filtering using Today, This week, This month, This year, All time, and Custom range options.

1.4 System Users

TechStar defines four primary user roles.

- **Supervisor:** The supervisor has executive-level access to service request monitoring, revenue analytics, staff account management, and technician workload capacity management. Supervisors can create and update Operator and Technician accounts.

-
- **Operator:** Operators manage front-desk service operations. Their responsibilities include creating service requests, searching customer records using phone numbers, assigning technicians, monitoring requests, and updating service progress.
 - **Technician:** Technicians perform repair work assigned to them. Their workload is tracked through `max_capacity`, `active_jobs`, specialization, and availability.
 - **Customer:** Customers can access the customer tracking interface using the Service Request ID and a 6-digit Customer Access Code. Customer tracking does not require application login.

1.5 System Assumptions

The following assumptions were considered during the analysis and design of the TechStar system:

1. **Service Request Creation:** Service requests are created by authorized workshop Operators or Supervisors rather than directly by customers.
2. **Customer Identification:** A customer's phone number is used as a primary means of identifying and searching customer records during service request creation and management.
3. **Technician Assignment:** A service request is assigned to an available Technician based on the Technician's availability, specialization, and current workload capacity.
4. **Service Request Lifecycle:** Each service request follows the predefined repair workflow and its status represents the current stage of the repair process.
5. **Progress Tracking:** Repair progress is represented using a percentage value ranging from 0% to 100% and is maintained together with the service request status.
6. **Staff Communication:** Real-time chat and notification features are intended for communication among authenticated workshop staff members.
7. **Technician Capacity:** A Technician's workload is constrained by the configured `max_capacity`, while `active_jobs` represents the number of currently active assigned jobs.

1.6 System Scope

TechStar focuses on managing the operational workflow of an electronics repair workshop. The system covers service request management, customer information and self-service tracking, technician assignment and workload management, staff account management, service progress and history tracking, real-time staff communication, notifications, background task processing, supervisory monitoring and analytics, and printable

service documents. The system does not perform the physical repair or component-level diagnosis of electronic devices. Direct credit card payment gateway hardware processing and third-party barcode scanner hardware integration are also outside the scope of the current system.

1.7 Functional Requirements

The functional requirements define the major capabilities provided by the TechStar system to support electronics repair workshop operations.

1.7.1 Service Request Management

- **Create Service Request:** Operators can create new service requests by recording customer information, device information, problem description, priority, and expected delivery date.
- **View Service Requests and Audit Logs:** Authorized staff can view service request lists and inspect the associated service request information and complete audit trail history.
- **Update Service Request Status and Progress:** Authorized staff can update the current repair status and adjust the service request progress using a value ranging from 0% to 100%.
- **Monitor Service Request Progress:** The system provides visual progress indicators and status timeline steps to represent the current state and progress of a repair request.
- **Cancel and Complete Service Requests:** Authorized staff can cancel active service requests or mark requests as completed. Completing a request supports recording the final payment amount for billing purposes.

1.7.2 Search and Filtering

The system provides search and filtering capabilities for service requests based on the following criteria:

- Service request status.
- Service request priority.
- Assigned technician.
- Customer phone number.
- Date range.

These filtering capabilities allow staff to locate relevant service requests and monitor requests according to operational requirements.

1.7.3 Staff Management

Supervisors can manage workshop staff accounts through the system.

The staff management functionality allows Supervisors to:

- Create Operator accounts.
- Create Technician accounts.
- Update Operator account details.
- Update Technician account details.

1.7.4 Real-Time Service Request Updates

The system uses Socket.IO to provide real-time synchronization of service request activities.

The real-time communication mechanism broadcasts events related to:

- Service request creation.
- Technician assignment.
- Service request status and progress updates.

This allows connected staff interfaces to receive service request changes without requiring manual page refreshes.

1.7.5 Staff Group Chat and Notifications

The system provides a real-time communication channel for authenticated staff members.

The functionality includes:

- Real-time staff group messaging.
- Delivery of chat messages through Socket.IO.
- Notification generation for relevant system events.
- User-specific notification tracking.
- Read and unread notification states.

1.8 Non-Functional Requirements

The non-functional requirements define the quality attributes and technical characteristics expected from the TechStar system.

1.8.1 Performance

The system targets API response times below 100 ms for standard CRUD operations. Optimized Supabase PostgreSQL queries and Socket.IO-based real-time event delivery are used to support responsive system interaction.

1.8.2 Scalability

The backend uses stateless JWT-based authentication and the non-blocking Node.js event-driven architecture. This architecture allows the backend API layer to be horizontally scaled when additional processing capacity is required.

1.8.3 Reliability

The system uses Graphile Worker as a PostgreSQL-backed background task queue for reliable execution of background operations. Fallback polling mechanisms are also used to improve the reliability of background task processing.

1.8.4 Security

The system applies multiple security mechanisms to protect authentication, authorization, and data access:

- **Password Hashing:** User passwords are hashed using bcrypt with a salt factor of 10.
- **JWT Authentication:** JSON Web Tokens are used to authenticate requests to protected API resources.
- **Role-Based Authorization:** The `authorizeRoles` middleware restricts access to operations according to the authenticated user's role.
- **Protected API Routes:** Authentication and authorization controls are applied to protected backend resources.

1.8.5 Maintainability

The backend follows a three-tier architecture consisting of Controllers, Services, and Repositories. This separation isolates HTTP request handling, business logic, and data access responsibilities.

The frontend follows a modular React component structure, allowing interface elements and application functionality to be developed and maintained independently.

1.8.6 Availability

The system uses Supabase-hosted PostgreSQL for persistent data storage and separates environment-specific configuration to support consistent application operation across development and deployment environments.

1.8.7 Usability

The frontend uses Tailwind CSS to provide a consistent and responsive user interface. Reusable interface elements include period-selection dropdowns, custom select fields, status badges, progress indicators, and other standardized controls.

1.8.8 Responsiveness

The user interface follows a mobile-first responsive design approach and is designed to provide usable layouts across mobile, tablet, and desktop screen sizes.

Chapter 2

System Design

2.1 System Architecture

TechStar follows a multi-tier architecture that separates the presentation layer, API layer, business logic, data access layer, database, real-time communication layer, and background processing layer.

The frontend communicates with the Express backend through REST APIs. The backend processes requests through authentication and authorization middleware before forwarding them to controllers, services, and repositories.

Supabase PostgreSQL provides persistent data storage. Socket.IO provides real-time communication, while Graphile Worker handles background tasks.

2.2 High-Level Architecture Diagram

Figure 2.1 presents the high-level architecture of the TechStar system.

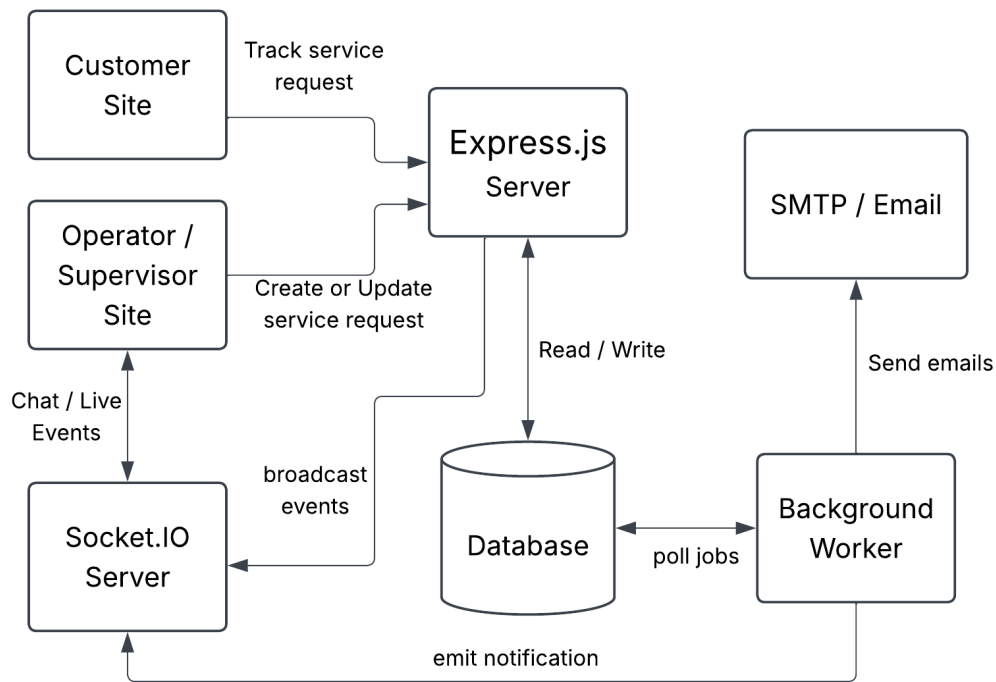


Figure 2.1: High-Level Architecture of TechStar

2.3 Database Design

TechStar uses Supabase PostgreSQL as its relational database.

The database consists of eight primary tables:

- **users:** Stores system user information, authentication-related information, roles, and timestamps.
- **technicians:** Stores technician-specific information such as specialization, work-load capacity, active jobs, and availability.
- **service_requests:** Stores repair requests, customer access information, customer and technician relationships, device information, problem descriptions, payment amounts, priority, status, progress, expected delivery time, and timestamps.
- **service_request_history:** Maintains the audit history of service request status and progress changes.
- **service_request_emails:** Tracks service-request related email processing status and email timestamps.
- **chat_messages:** Stores staff chat messages and their associated sender and creation timestamp.

- `notifications`: Stores notification records associated with service requests or chat messages.
- `user_notification_track`: Maintains user-specific notification read status.

2.3.1 Database Schema

The principal attributes of the database tables are summarized below.

Table 2.1: TechStar Database Tables

Table	Principal Attributes
<code>users</code>	<code>id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>password_hash</code> , <code>role</code> , <code>created_at</code> , <code>updated_at</code>
<code>technicians</code>	<code>id</code> , <code>user_id</code> , <code>specialization</code> , <code>max_capacity</code> , <code>active_jobs</code> , <code>is_available</code> , <code>created_at</code> , <code>updated_at</code>
<code>service_requests</code>	<code>id</code> , <code>customer_access_code</code> , <code>customer_id</code> , <code>technician_id</code> , <code>created_by</code> , <code>device_info</code> , <code>problem_description</code> , <code>payment_amount</code> , <code>priority</code> , <code>status</code> , <code>progress</code> , <code>expected_delivery_at</code> , <code>created_at</code> , <code>updated_at</code> , <code>completed_at</code>
<code>service_request_history</code>	<code>id</code> , <code>service_request_id</code> , <code>changed_by</code> , <code>old_status</code> , <code>new_status</code> , <code>old_progress</code> , <code>new_progress</code> , <code>note</code> , <code>created_at</code>
<code>service_request_emails</code>	<code>service_id</code> , <code>status</code> , <code>email_sent_at</code>
<code>chat_messages</code>	<code>id</code> , <code>sender_id</code> , <code>content</code> , <code>created_at</code>
<code>notifications</code>	<code>id</code> , <code>service_request_id</code> , <code>chat_message_id</code> , <code>type</code> , <code>message</code> , <code>created_at</code>
<code>user_notification_track</code>	<code>id</code> , <code>notification_id</code> , <code>user_id</code> , <code>is_read</code> , <code>created_at</code>

2.4 Entity-Relationship Diagram

Figure 2.2 presents the relationships between the database entities.

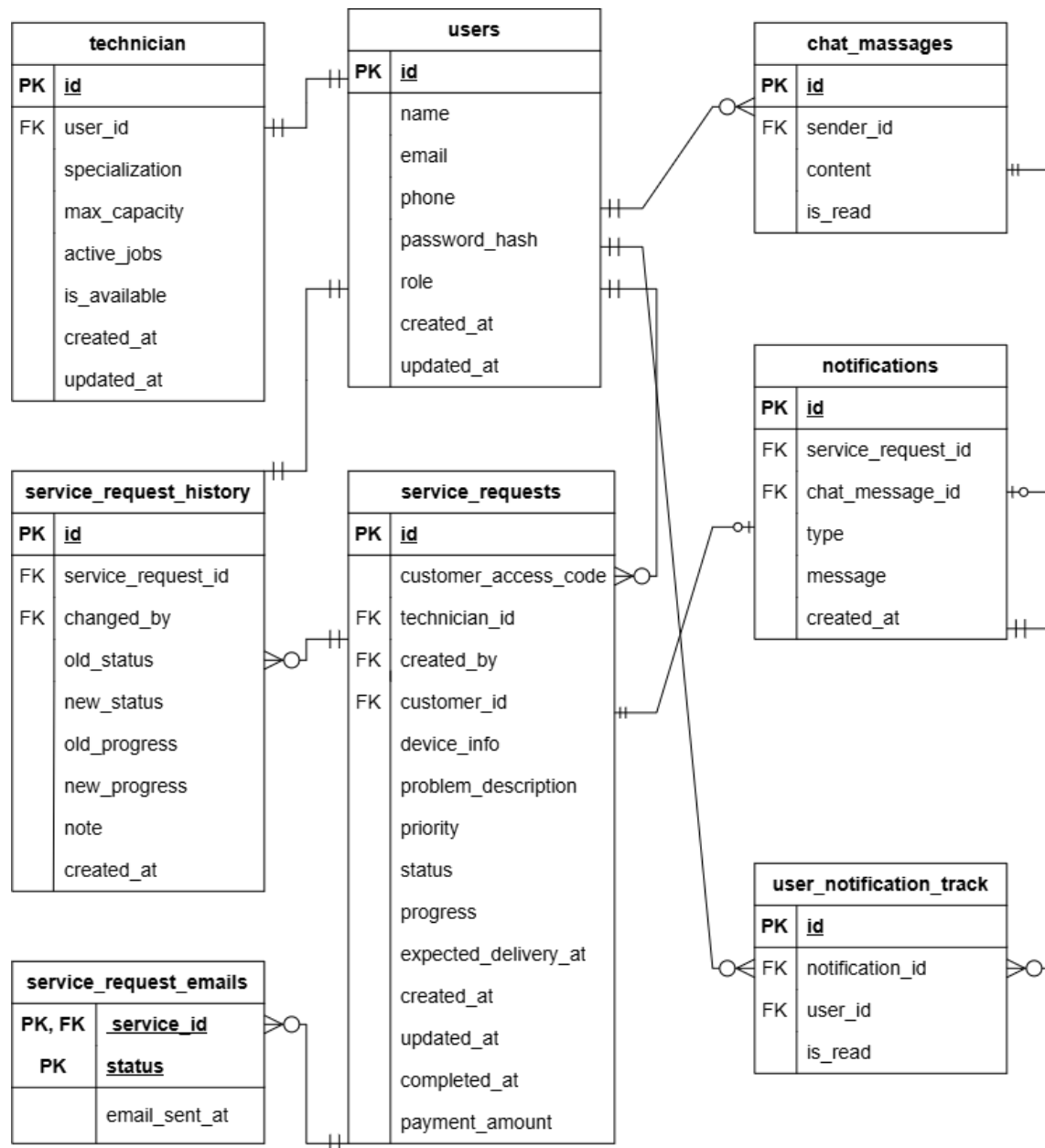


Figure 2.2: Entity-Relationship Diagram of TechStar

2.5 API Design

TechStar exposes RESTful API endpoints through the Express backend. All API endpoints are prefixed with /api. Protected endpoints require authentication using the HTTP Authorization header with the Bearer token scheme. Role-based authorization is applied to endpoints according to the authenticated user's role.

The API is organized into authentication, service request, user and staff management, technician management, supervisory analytics, notification, and staff communication resources. Table 2.3 summarizes the implemented REST API endpoints.

Table 2.2: TechStar REST API Endpoints

Method	Endpoint	Purpose	Access
POST	/api/auth/login	Authenticate a user and obtain a JWT access token	Public
POST	/api/auth/refresh	Refresh the JWT access token	Public
POST	/api/auth/logout	End the current user session	Public
GET	/api/auth/me	Retrieve the authenticated user's profile	Authenticated
GET	/api/service-requests/track	Track a service request using its ID and customer access code	Public
GET	/api/service-requests	Retrieve service request records	Operator/Supervisor
POST	/api/service-requests	Create a new service request	Operator/Supervisor
GET	/api/service-requests/:id	Retrieve service request details	Operator/Supervisor
GET	/api/service-requests/:id/history	Retrieve the service request audit history	Operator/Supervisor
PATCH	/api/service-requests/:id/assign	Assign a technician to a service request	Operator/Supervisor
PATCH	/api/service-requests/:id/status	Update the service request status	Operator/Supervisor
PATCH	/api/service-requests/:id/progress	Update the repair progress percentage	Operator/Supervisor
PATCH	/api/service-requests/:id/complete	Mark a service request as completed	Operator/Supervisor
PATCH	/api/service-requests/:id/cancel	Cancel a service request	Operator/Supervisor
PATCH	/api/service-requests/:id	Update service request details	Operator/Supervisor
GET	/api/users	Retrieve system users	Supervisor/Operator
GET	/api/users/customers	Find a customer profile by phone number	Supervisor/Operator
POST	/api/users/customers	Create a customer account	Supervisor/Operator
POST	/api/users/operators	Create an Operator account	Supervisor
POST	/api/users/technicians	Create a Technician account and workload profile	Supervisor
PATCH	/api/users/:id	Update a user profile and account details	Supervisor/Operator
GET	/api/technicians	Retrieve technicians with active job counts	Operator/Supervisor © TechStar
GET	/api/technicians/available	Retrieve available technicians	Operator/Supervisor

Table 2.3: TechStar REST API Endpoints

Method	Endpoint	Purpose	Access
GET	<code>/api/technicians/:id</code>	Retrieve technician details	Operator/Supervisor
PATCH	<code>/api/technicians/:id</code>	Update technician capacity, specialization, and availability	Supervisor
GET	<code>/api/supervisor/overview</code>	Retrieve supervisory overview, revenue, and analytics	Supervisor
GET	<code>/api/notifications</code>	Retrieve user notifications	Authenticated
GET	<code>/api/notifications/unread-count</code>	Retrieve the number of unread notifications	Authenticated
PATCH	<code>/api/notifications/:id/read</code>	Mark a notification as read	Authenticated
POST	<code>/api/notifications/read-all</code>	Mark all user notifications as read	Authenticated
GET	<code>/api/chat/messages</code>	Retrieve recent staff group chat messages	Authenticated
POST	<code>/api/chat/messages</code>	Send a staff group chat message	Authenticated

2.6 WebSocket Communication

TechStar uses Socket.IO to provide real-time bidirectional communication between the frontend and backend. WebSocket communication is primarily used for service request synchronization, technician assignment coordination, staff group chat, and notification delivery.

2.6.1 Client-to-Server Events

Connected clients emit the following events:

- **join:** Joins a specified room channel using a room identifier, such as `serviceRequest:1`.
- **joinUser:** Joins the authenticated user's personal channel in the form `user:<userId>`.
- **joinRole:** Joins a role-based channel such as operators or supervisors.
- **joinServiceRequest:** Joins the real-time communication room associated with a particular service request.
- **technician:reservation:** Temporarily reserves or releases a technician during assignment using technician, request, and reservation information.
- **chat:send:** Sends a staff group chat message to the backend.

2.6.2 Server-to-Client Events

The backend broadcasts or emits the following events:

- **serviceRequest:created:** Emitted when a new service request is created, including the service request and corresponding history information.
- **serviceRequest:assigned:** Emitted when a technician is assigned to a service request.
- **serviceRequest:statusUpdated:** Emitted when the status or progress of a service request is updated.
- **serviceRequest:historyCreated:** Emitted when a new audit history entry is created.
- **technician:reservationChanged:** Emitted when a technician reservation is created, changed, or released.
- **chat:message:** Emitted to connected staff clients when a new group chat message is sent.
- **chat:unread:** Emitted to staff clients to indicate unread chat activity.
- **notification:new:** Emitted to the targeted user's personal socket room when a new notification is generated.

2.7 Background Processing

TechStar uses Graphile Worker to execute background tasks independently from the main HTTP request-response cycle. This mechanism allows periodic monitoring and email-related operations to be processed asynchronously.

- **Overdue Repair Detector:** The overdue-check background task periodically inspects active service requests. When a request exceeds its `expected_delivery_at` time, an OVERDUE notification is generated and broadcast to the relevant user and supervisors.
- **Email Confirmation Worker:** The email worker sends customer confirmation emails when service requests are created or when their status changes to `READY_FOR_DELIVERY` or `FAILED`.

Using Graphile Worker separates background processing from the main API execution path and allows these tasks to be processed asynchronously through the PostgreSQL-backed job queue.

2.8 Technology Stack

The primary technologies used by TechStar are:

Table 2.4: Technology Stack

Technology	Purpose
React 18 + Vite	Frontend application development and build tooling
Tailwind CSS	Responsive user-interface styling
Node.js	Backend runtime environment
Express.js	REST API and server-side application framework
Socket.IO	Real-time bidirectional communication
Supabase PostgreSQL	Relational database and persistent data storage
Graphile Worker	PostgreSQL-backed background task processing
JWT	Stateless authentication
bcrypt	Password hashing

2.9 Technology Selection Rationale

- **React 18 + Vite:** React supports component-based frontend development, while Vite provides fast development and efficient builds.
- **Tailwind CSS:** Tailwind CSS enables responsive UI development through a utility-first styling approach.
- **Node.js + Express:** Node.js provides an event-driven runtime for API operations, while Express provides a lightweight framework for building REST APIs.
- **Socket.IO:** Socket.IO enables real-time bidirectional communication between clients and the server.
- **Supabase PostgreSQL:** Supabase provides a managed PostgreSQL database with relational constraints, indexing, and database RPC support.
- **Graphile Worker:** Graphile Worker provides reliable PostgreSQL-backed background task processing for operations such as email delivery and overdue request detection.

Chapter 3

Implementation

3.1 Implementation Overview

The implementation follows the architectural design presented in the previous chapter. The backend separates API routing, request handling, business logic, database access, real-time communication, and background processing into dedicated modules. The frontend is organized around reusable React components, application contexts, page-level views, API services, and Socket.IO communication.

The major backend modules include configuration, routes, controllers, middleware, services, repositories, sockets, tasks, and workers. The frontend consists of reusable components, context providers, application pages, API service modules, and Socket.IO clients.

3.2 Project Directory Structure

The project is divided into two primary applications: backend and frontend. The backend contains the REST API, database scripts, real-time communication, and background processing components. The frontend contains the user interfaces and client-side communication modules.

The major project structure is shown below.

```
TechStar/  
  backend/  
    sql/  
      schema.sql  
      seed.sql  
    src/  
      config/  
      controllers/  
      middleware/  
      repositories/  
      routes/
```

```
    services/  
    sockets/  
    tasks/  
    workers/  
package.json  
README.md  
  
frontend/  
  public/  
  src/  
    assets/  
    components/  
    context/  
    pages/  
    services/  
    sockets/  
package.json  
vite.config.js  
  
README.md
```

3.3 Backend Structure

The backend follows a layered structure in which each module has a specific responsibility. This separation improves maintainability and keeps request handling, business logic, and database operations independent.

- **Config:** Contains environment configuration, database connection setup, and database migration-related files.
- **Routes:** Defines the REST API endpoints and connects incoming requests to the appropriate middleware and controllers.
- **Controllers:** Handles HTTP requests, validates request-level input, invokes application services, and returns HTTP responses.
- **Middleware:** Provides cross-cutting request processing such as JWT authentication, role-based authorization, error handling, and rate limiting.
- **Services:** Contains the application's business logic, including service request processing, authentication, notifications, chat, supervisor operations, and email-related functionality.
- **Repositories:** Provides the database access layer for users, service requests, technicians, notifications, chat messages, and history records.
- **Sockets:** Implements Socket.IO connection handling and real-time service request events.

-
- **Tasks:** Contains background task definitions for operations such as overdue request checking and service request email processing.
 - **Workers:** Runs the Graphile Worker process responsible for executing background tasks from the PostgreSQL-backed job queue.

3.4 Frontend Structure

The frontend follows a component-based React architecture. The application separates reusable interface elements from page-level views and communication logic.

- **Assets:** Contains application images and graphical resources.
- **Components:** Contains reusable interface components grouped by purpose, including authentication, common UI elements, layouts, service request components, and supervisor-specific components.
- **Context:** Provides application-level state management through React Context, including authentication and period-based filtering.
- **Pages:** Contains page-level views for authentication, customer tracking, operator operations, supervisor operations, and staff chat.
- **Services:** Contains client-side API modules used to communicate with the backend REST API.
- **Sockets:** Contains the Socket.IO client connection and real-time event handling modules.

3.5 Database Implementation

The database layer is implemented using PostgreSQL through Supabase. The database schema defines the application's relational structure, constraints, indexes, and database functions required by the system.

The `schema.sql` file contains the primary database definitions, while `seed.sql` provides initial data for development and demonstration purposes. Database-related migration files are maintained separately to support controlled schema changes.

The repository layer provides an abstraction between the application services and the database. This allows business logic to remain independent of direct database operations.

3.6 Authentication and Authorization

Authentication is implemented using JSON Web Tokens (JWT). After successful login, the server issues a JWT that is used to identify the authenticated user when accessing protected API resources.

Authorization is implemented through role-based middleware. The middleware verifies the authenticated user's role before allowing access to restricted operations.

The system supports role-specific access for staff users, including operators, technicians, and supervisors. Supervisor-level operations include staff account management and monitoring functions.

3.7 Customer Access Control

Customer tracking is implemented using a service request identifier and a unique six-digit customer access code. The mechanism allows customers to retrieve the status and relevant information of a service request without requiring an authenticated staff account.

The customer access mechanism is intentionally separated from the staff authentication system so that customers can track their repair requests through the public tracking interface.

3.8 Real-Time Communication

Real-time communication is implemented using Socket.IO. The backend Socket.IO handlers emit service request events when relevant changes occur, allowing connected clients to update their interfaces without requiring continuous manual refreshes.

The frontend maintains the Socket.IO connection and provides client-side event handlers for receiving real-time updates.

This mechanism is primarily used for service request synchronization and staff communication.

3.9 Background Processing

TechStar uses Graphile Worker for asynchronous background processing. Background tasks are executed separately from the main HTTP request-response cycle.

The implemented background tasks include:

- **Overdue Detection:** Checks service requests and identifies requests that have exceeded their expected completion time.
- **Service Request Email Processing:** Processes queued email tasks for service request-related notifications.

Graphile Worker uses PostgreSQL as the persistent job queue, allowing background tasks to be processed independently from the main API server.

3.10 API and Service Communication

The frontend communicates with the backend through REST API service modules. Separate client-side service modules are maintained for authentication-related operations, service requests, users, technicians, supervisors, notifications, and chat functionality.

The backend routes map these API requests to controllers, while controllers delegate application operations to the appropriate services. Services then interact with repositories when database operations are required.

This structure provides a clear flow from the frontend request to the API endpoint, business logic, and database layer.

3.11 Running the Application

3.11.1 Backend

The backend dependencies can be installed and the development server can be started using:

```
cd backend
npm install
npm run dev
```

The required environment variables must be configured before starting the backend.

3.11.2 Frontend

The frontend dependencies can be installed and the development server can be started using:

```
cd frontend
npm install
npm run dev
```

The frontend environment configuration must contain the required backend API and Socket.IO connection settings.

3.12 Implementation Summary

The final implementation integrates the frontend, REST API, PostgreSQL database, real-time communication, authentication, authorization, and background processing into a unified repair workshop management platform.

The implemented system provides service request management, role-based access control, customer self-service tracking, technician workload management, real-time updates, staff communication, notifications, supervisor monitoring, analytics, and asynchronous background processing.

Bibliography

- [1] React Documentation. <https://react.dev/>
- [2] Express.js Documentation. <https://expressjs.com/>
- [3] Socket.IO Documentation. <https://socket.io/>
- [4] Supabase Documentation. <https://supabase.com/docs>
- [5] GraphQL Worker Documentation. <https://worker.graphile.org/>